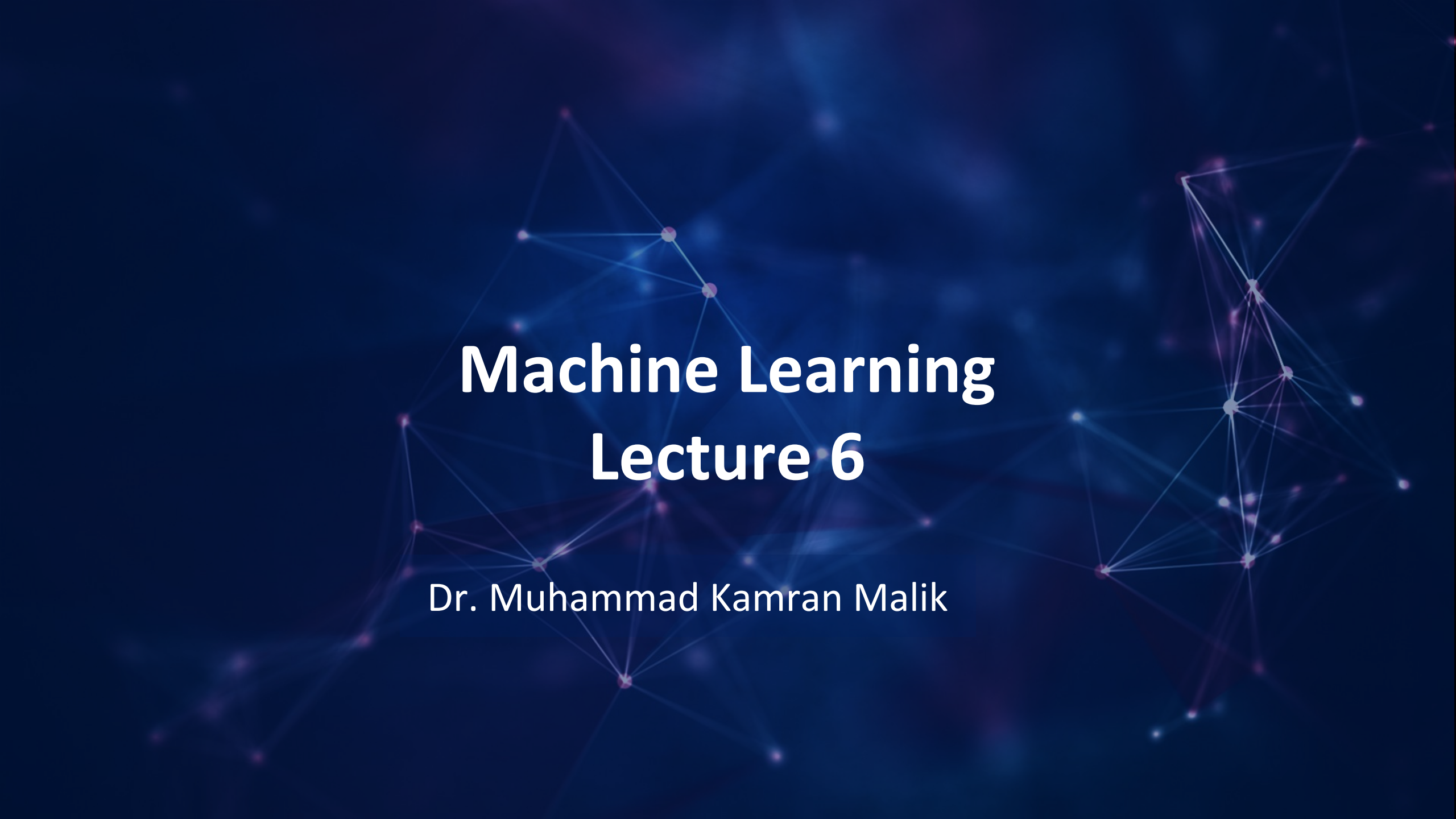




# Machine Learning Lecture 6

Dr. Muhammad Kamran Malik



# Linear Regression with Multiple Variables

## Multiple Features

# Multiple features (variables)

one  
feature



| Size in feet <sup>2</sup> ( $x$ ) | Price (\$) in 1000's ( $y$ ) |
|-----------------------------------|------------------------------|
| 2104                              | 400                          |
| 1416                              | 232                          |
| 1534                              | 315                          |
| 852                               | 178                          |
| ...                               | ...                          |



$$f_{w,b}(x) = wx + b$$

# Multiple features (variables)

|       | Size in<br>feet <sup>2</sup><br>$x_1$ | Number of<br>bedrooms<br>$x_2$ | Number of<br>floors<br>$x_3$ | Age of home<br>in years<br>$x_4$ | Price (\$) in<br>\$1000's |
|-------|---------------------------------------|--------------------------------|------------------------------|----------------------------------|---------------------------|
|       | 2104                                  | 5                              | 1                            | 45                               | 460                       |
| $i=2$ | 1416                                  | 3                              | 2                            | 40                               | 232                       |
|       | 1534                                  | 3                              | 2                            | 30                               | 315                       |
|       | 852                                   | 2                              | 1                            | 36                               | 178                       |
|       | ...                                   | ...                            | ...                          | ...                              | ...                       |

$j = 1 \dots 4$   
 $n = 4$

$x_j$  =  $j^{\text{th}}$  feature

$n$  = number of features

$\vec{x}^{(i)}$  = features of  $i^{\text{th}}$  training example

$x_j^{(i)}$  = value of feature  $j$  in  $i^{\text{th}}$  training example

•  $\vec{x}^{(2)} = [1416 \ 3 \ 2 \ 40]$

$x_3^{(2)} = 2$

# Model:

Previously:  $f_{w,b}(x) = wx + b$

example

$$f_{w,b}(X) = w_1 X_1 + w_2 X_2 + w_3 X_3 + w_4 X_4 + b$$
$$f_{w,b}(X) = 0.1 \underset{\substack{\uparrow \\ \text{size}}}{X_1} + \underset{\substack{\bullet \\ \uparrow \\ \text{\# bedrooms}}}{4} X_2 + \underset{\substack{\bullet \\ \uparrow \\ \text{\# floors}}}{10} X_3 + \underset{\substack{\bullet \\ \uparrow \\ \text{years}}}{-2} X_4 + \underset{\substack{\uparrow \\ \text{base price}}}{80}$$

$$f_{w,b}(\mathbf{x}) = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$$

# Model:

$$f_{w,b}(\vec{x}) = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$$

$$\vec{w} = [w_1 \ w_2 \ w_3 \ \dots \ w_n]$$

parameters  
of the model

$b$  is a number

vector  $\vec{x} = [x_1 \ x_2 \ x_3 \ \dots \ x_n]$

$$f_{w,b}(\vec{x}) = \vec{w} \cdot \vec{x} + b = w_1 x_1 + w_2 x_2 + w_3 x_3 + \dots + w_n x_n + b$$

dot product

multiple linear regression

(not multivariate regression)



# Vectorization

## Parameters and features

$$\vec{w} = [w_1 w_2 w_3] \quad n=3$$

$b$  is a number

$$\vec{x} = [x_1 x_2 x_3]$$

linear algebra: count from 1

$w[0]$   $w[1]$   $w[2]$

NumPy 

```
w =  
np.array([1.0, 2.5, -1.3])  
b = 4
```

```
x = np.array([10, 20, 30])
```

## Without vectorization $n=100,000$

$$f_{\vec{w},b}(\vec{x}) = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

```
f = w[0] * x[0] +  
     w[1] * x[1] +  
     w[2] * x[2] + b
```



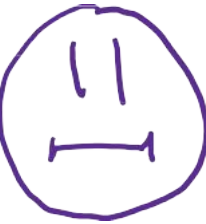
## Without vectorization

$$f_{\vec{w},b}(\vec{x}) = \left( \sum_{j=1}^n w_j x_j \right) + b$$

$\sum_{j=1}^n \rightarrow j=1 \dots n$   
 $1, 2, 3$

$\text{range}(0, n) \rightarrow j=0 \dots n-1$

```
f = 0  
for j in range(n):  
    f = f + w[j] * x[j]  
f = f + b
```



## Vectorization

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

```
f = np.dot(w, x) + b
```



## Without vectorization

```
for j in range(0,16):  
    f = f + w[j] * x[j]
```

$t_0$   $f + w[0] * x[0]$

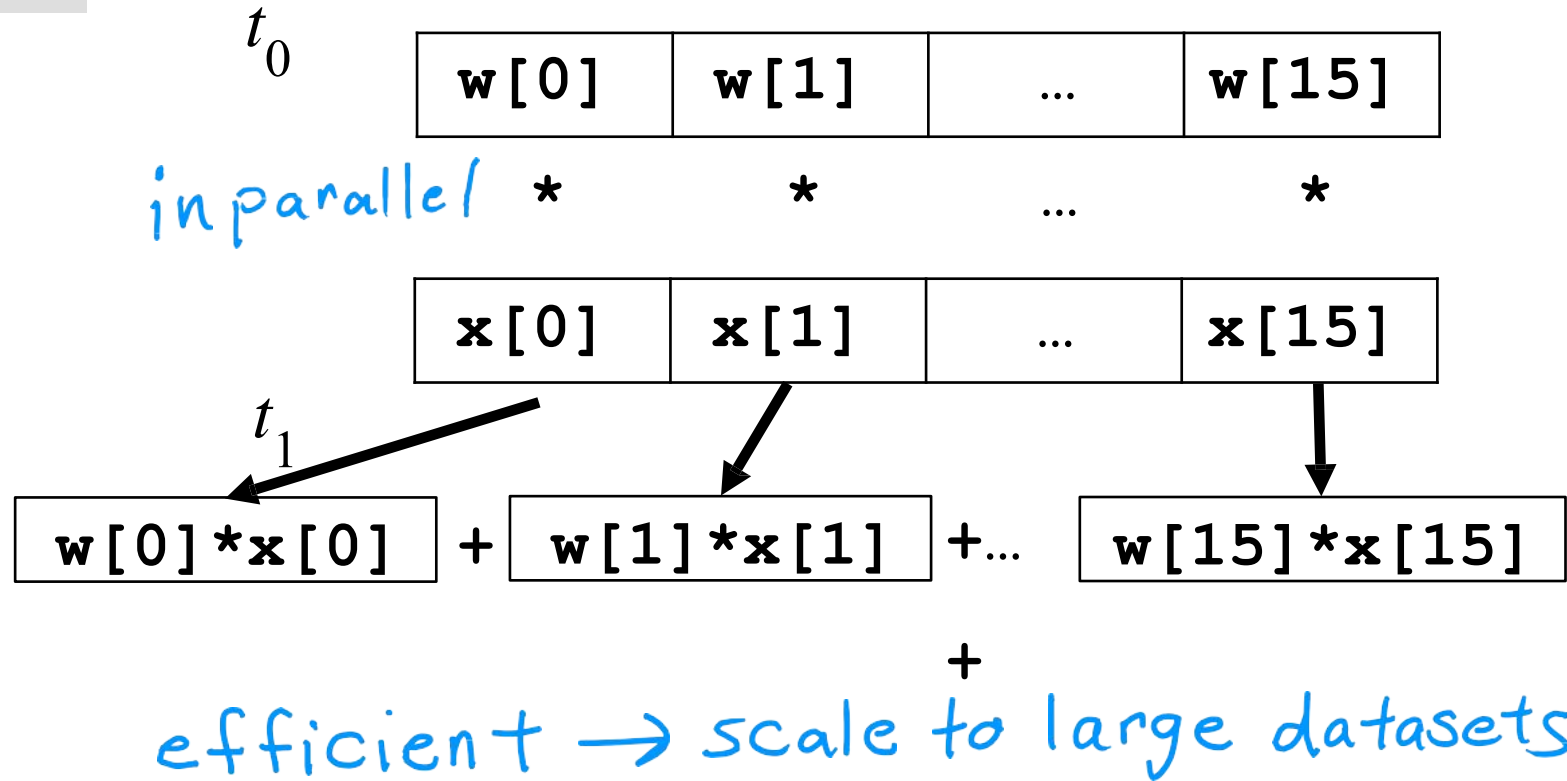
$t_1$   $f + w[1] * x[1]$

...

$t_{15}$   $f + w[15] * x[15]$

## Vectorization

```
np.dot(w,x)
```



Gradient descent

*derivatives*  $\vec{w} = \begin{pmatrix} w_1 & w_2 & \dots & w_{16} \\ d_1 & d_2 & \dots & d_{16} \end{pmatrix}$  ~~b~~ *parameters*

```
w = np.array([0.5, 1.3, ... 3.4])
```

```
d = np.array([0.3, 0.2, ... 0.4])
```

*learning rate  $\alpha$*   
compute  $w_j = w_j - 0.1d_j$  for  $j = 1 \dots 16$

Without vectorization

$$w_1 = w_1 - 0.1d_1$$

$$w_2 = w_2 - 0.1d_2$$

$\vdots$

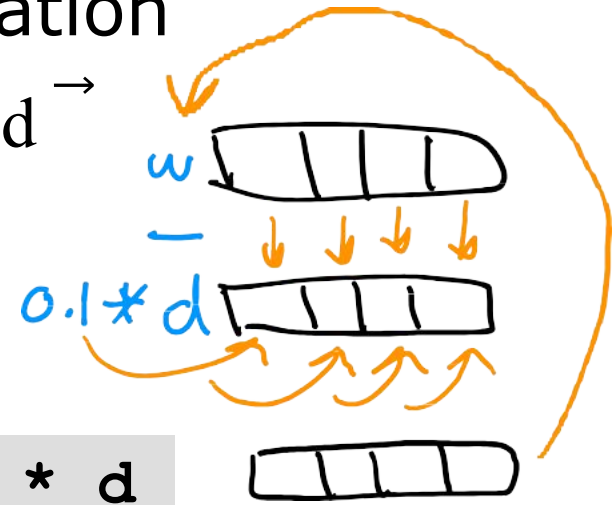
$$w_{16} = w_{16} - 0.1d_{16}$$

```
for j in range(0,16):
```

```
    w[j] = w[j] - 0.1 * d[j]
```

With vectorization

$$\vec{w} = \vec{w} - 0.1\vec{d}$$



```
w = w - 0.1 * d
```



# Gradient Descent for Multiple Regression

## Previous notation

Parameters

$$w_1, \dots, w_n$$
$$b$$

Model  $f_{w,b}(\vec{x}) = w_1 x_1 + \dots + w_n x_n + b$

Cost function  $J(w_1, \dots, w_n, b)$

Gradient descent

repeat

$$\left\{ \begin{array}{l} w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(w_1, \dots, w_n, b) \\ b = b - \alpha \frac{\partial}{\partial b} J(w_1, \dots, w_n, b) \end{array} \right\}$$

## Vector notation

vector of length  $n$

$$\vec{w} = [w_1 \dots w_n]$$

$b$  still a number

$$f_{w,b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

dot product

$$J(\vec{w}, b)$$

repeat

$$\left\{ \begin{array}{l} w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b) \\ b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b) \end{array} \right\}$$

# Gradient descent

One feature

repeat {

$$\underline{w} = w - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) \underline{x^{(i)}}$$

$$\searrow \frac{\partial}{\partial w} J(w, b)$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

simultaneously update  $w, b$

}

$n$  features ( $n \geq 2$ )

repeat {

$$\overset{j=1}{\underline{w_1}} = w_1 - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\bar{w},b}(\bar{x}^{(i)}) - y^{(i)}) \underline{x_1^{(i)}}$$

$\vdots$

$\overset{j=n}{w_n}$

$$w_n = w_n - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\bar{w},b}(\bar{x}^{(i)}) - y^{(i)}) x_n^{(i)}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\bar{w},b}(\bar{x}^{(i)}) - y^{(i)})$$

simultaneously update

$w_j$  (for  $j = 1, \dots, n$ ) and  $b$

}

# An alternative to gradient descent

## Normal equation

- Only for linear regression
- Solve for  $w$ ,  $b$  without iterations

## Disadvantages

- Doesn't generalize to other learning algorithms.
- Slow when number of features is large ( $> 10,000$ )

## What you need to know

- Normal equation method may be used in machine learning libraries that implement linear regression.
- Gradient descent is the recommended method for finding parameters  $w, b$



# Feature Scaling

# Feature and parameter values

$$\text{price} = w_1 x_1 + w_2 x_2 + b$$

$x_1$ : size (feet<sup>2</sup>) range: 300 – 2,000 ● large  
 $x_2$ : # bedrooms range: 0 – 5 small

size  $\downarrow$   $x_1$   $\downarrow$   $x_2$   
 bedrooms

House:  $x_1 = 2000$ ,  $x_2 = 5$ ,  $\text{price} = \$500\text{k}$  one training example

size of the parameters  $w_1, w_2$ ?

$$w_1 = 50, w_2 = 0.1, b = 50 \quad w_1 = 0.1, w_2 = 50, b = 50$$

small large

$$\text{price} = 50 * 2000 + 0.1 * 5 + 50$$

$\text{price} = \$100,050.5\text{k}$

$$= 0.1 * \frac{2000\text{k}}{200\text{k}} + 50 * \frac{5}{250\text{k}} + 50$$

$\text{price} = \$500\text{k}$

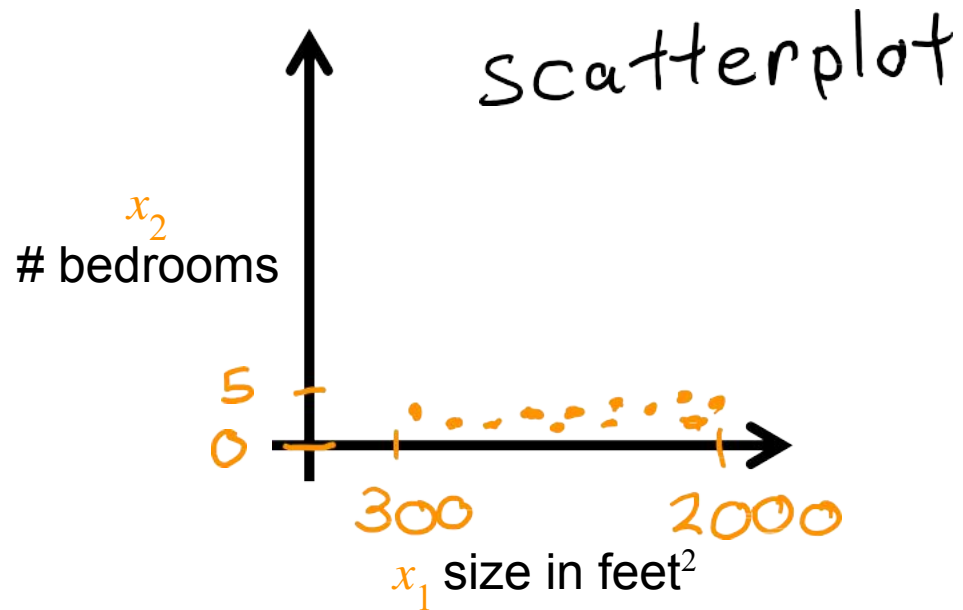
$\text{price} = \$100,050.5\text{k}$

more reasonable

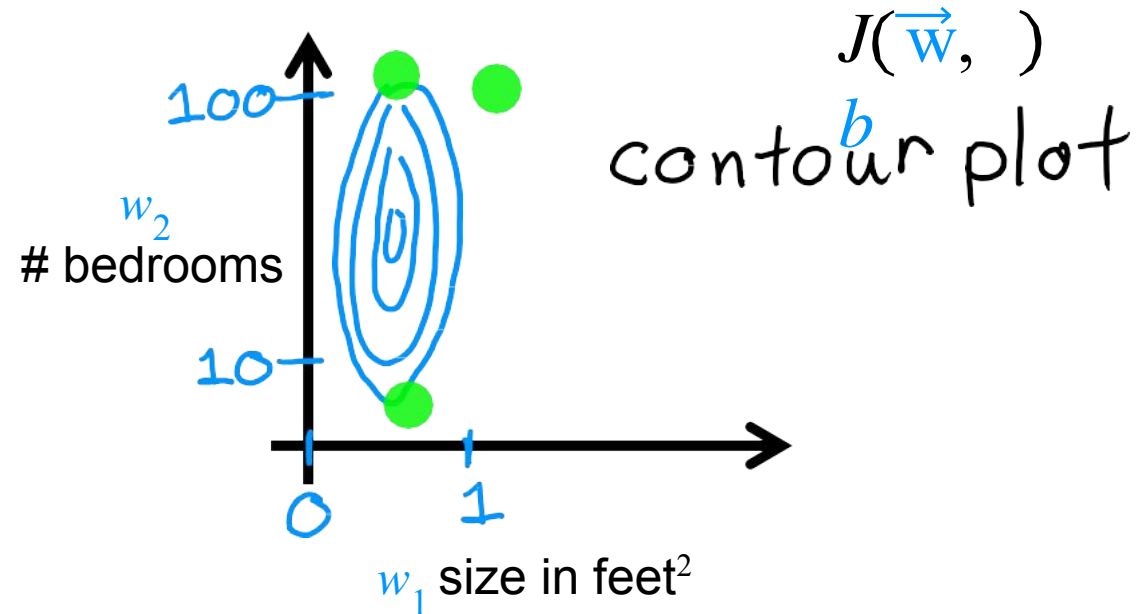
# Feature size and parameter size

|                           | size of feature $x_j$ | size of parameter $w_j$ |
|---------------------------|-----------------------|-------------------------|
| size in feet <sup>2</sup> | ←→                    | ←→                      |
| #bedrooms                 | ←→                    | ←→                      |

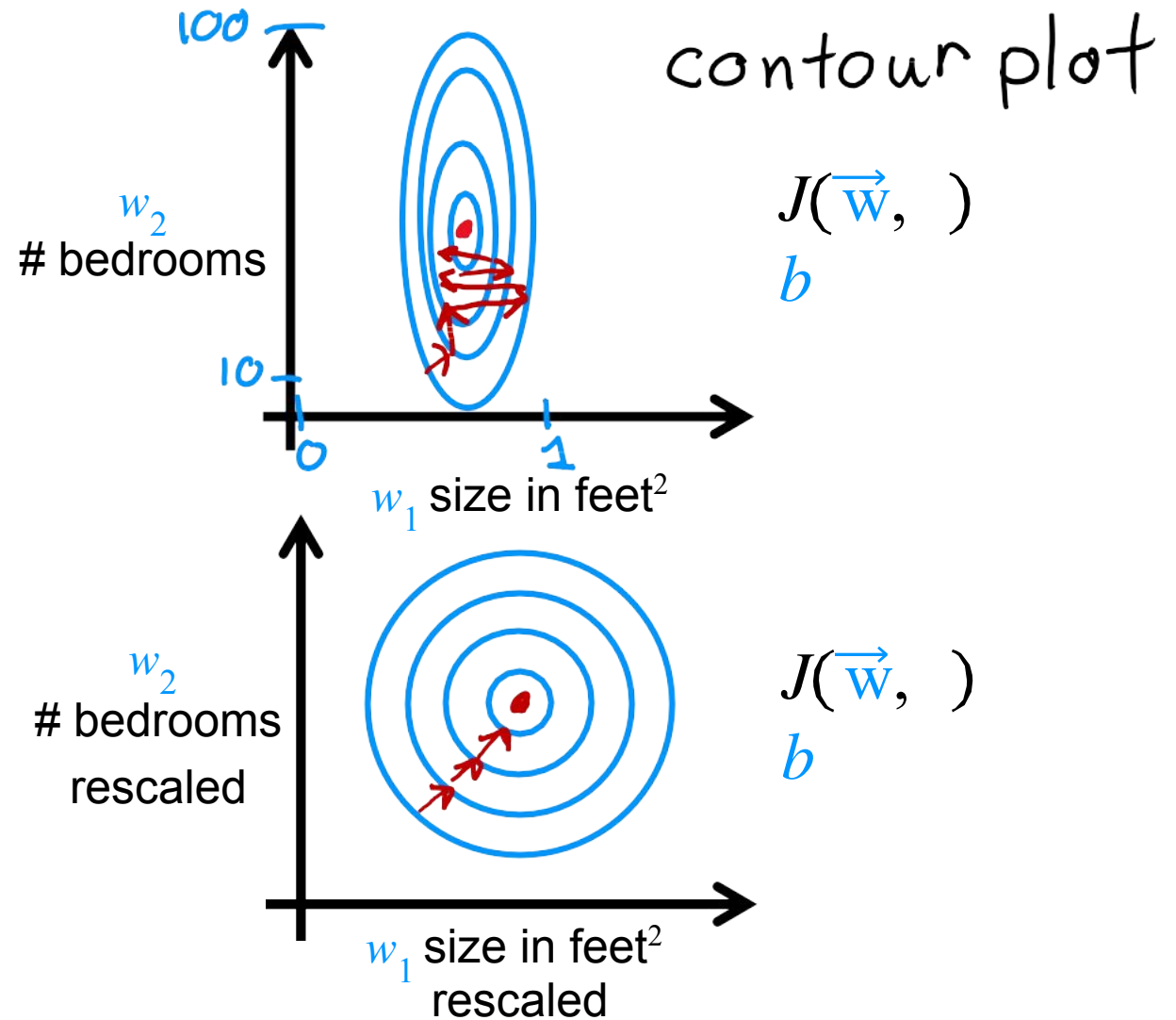
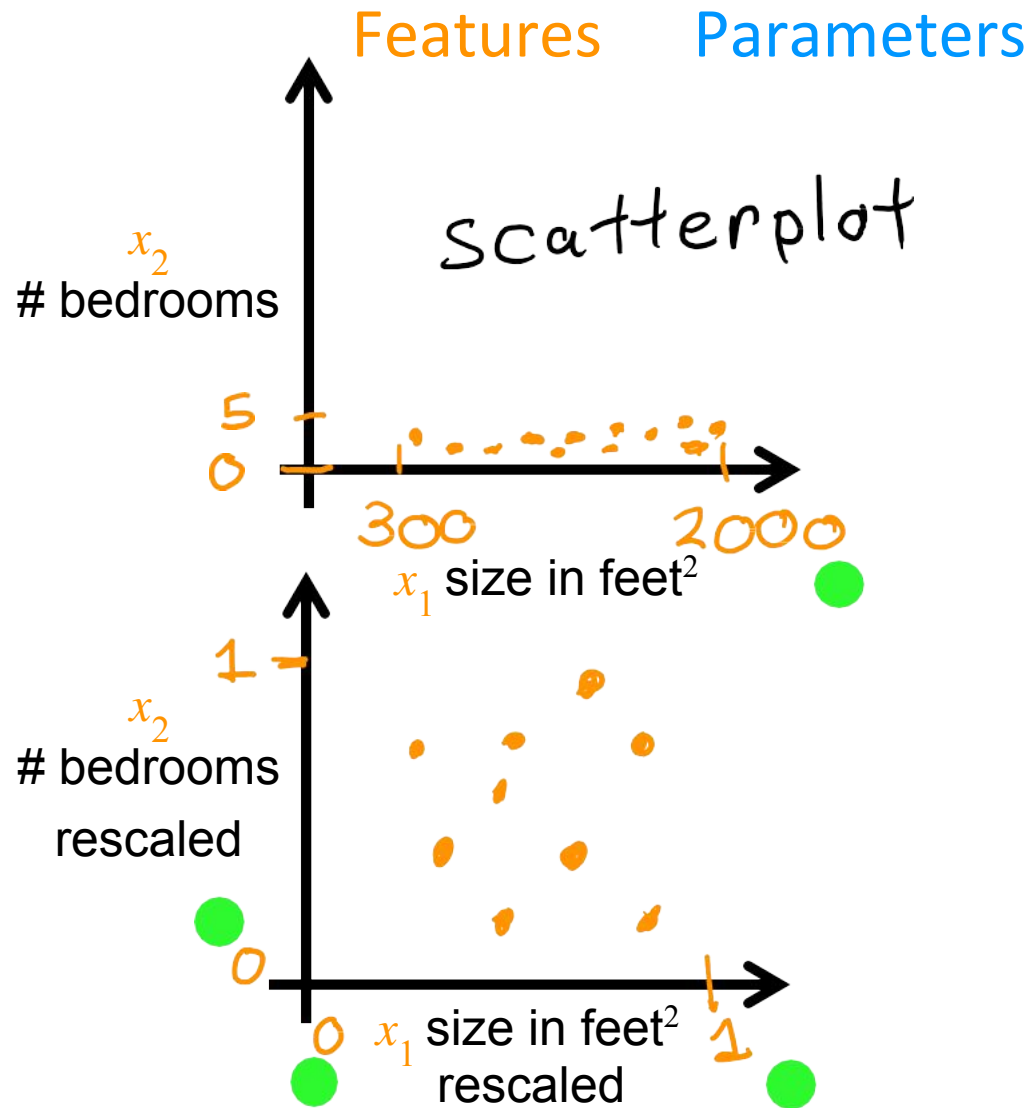
## Features



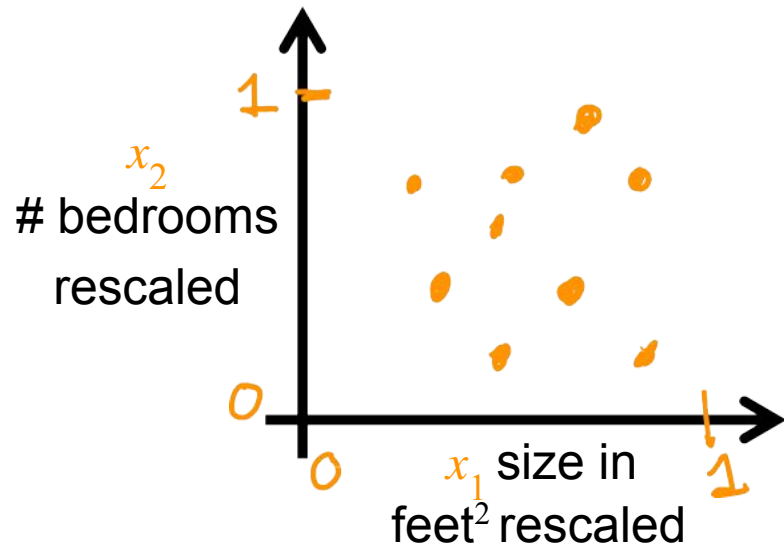
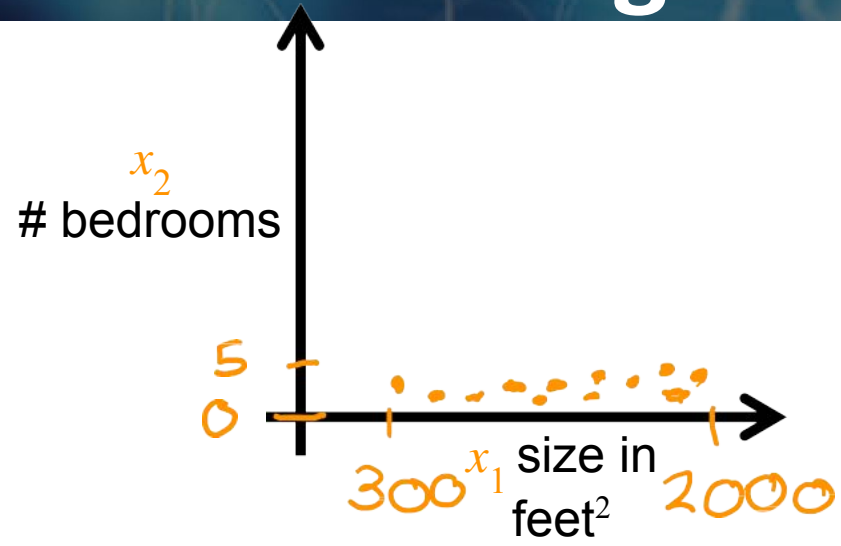
## Parameters



# Feature size and gradient descent



# Feature scaling



$$300 \leq x_1 \leq 2000$$

$$x_{1,scaled} = \frac{x}{2000}$$

max

$$0.15 \leq x_{1,scaled} \leq 1$$

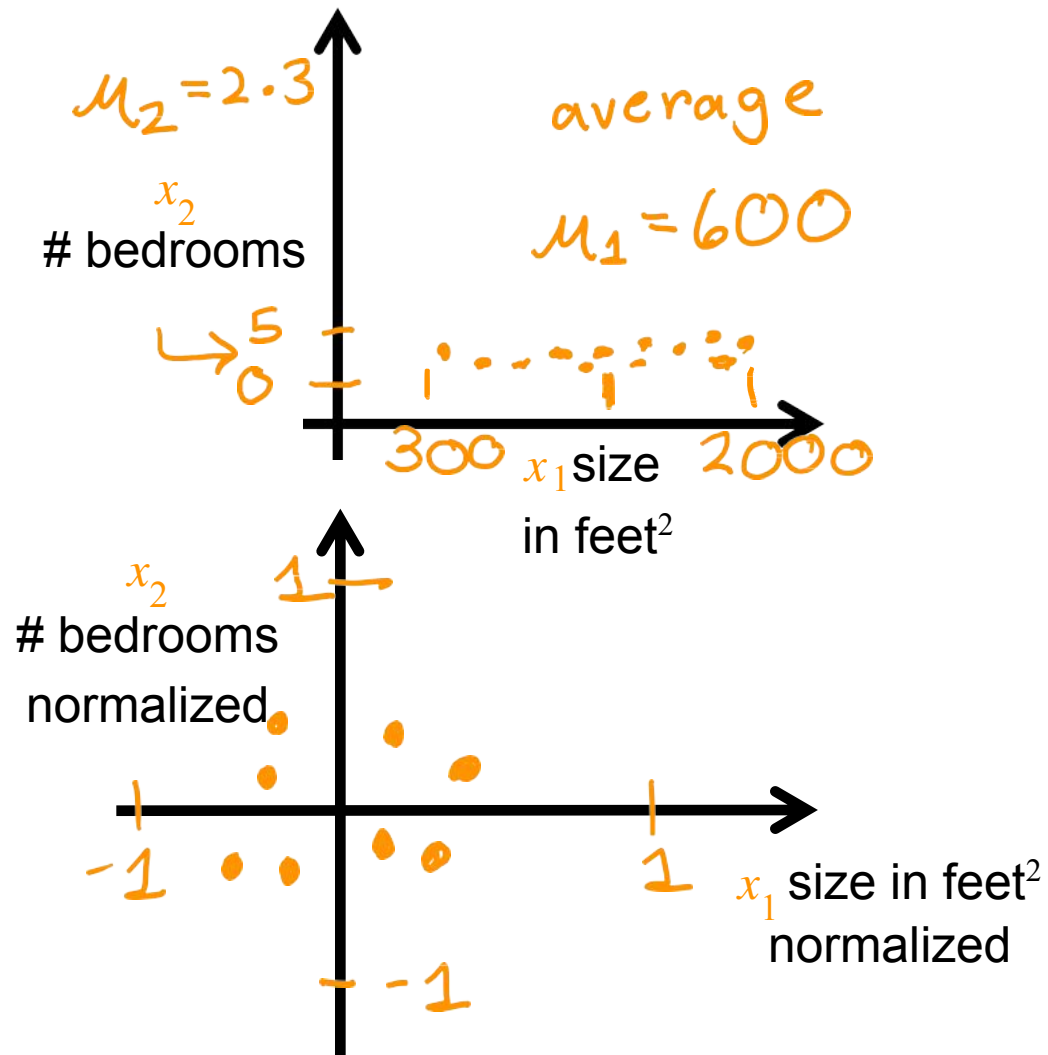
$$0 \leq x_2 \leq 5$$

$$x_{2,scaled} = \frac{x}{5}$$

max

$$0 \leq x_{2,scaled} \leq 1$$

# Mean normalization



$$300 \leq x_1 \leq 2000$$

$$x_1 = \frac{x_1 - \mu_1}{2000 - 300}$$

max-min

$$-0.18 \leq x_1 \leq 0.82$$

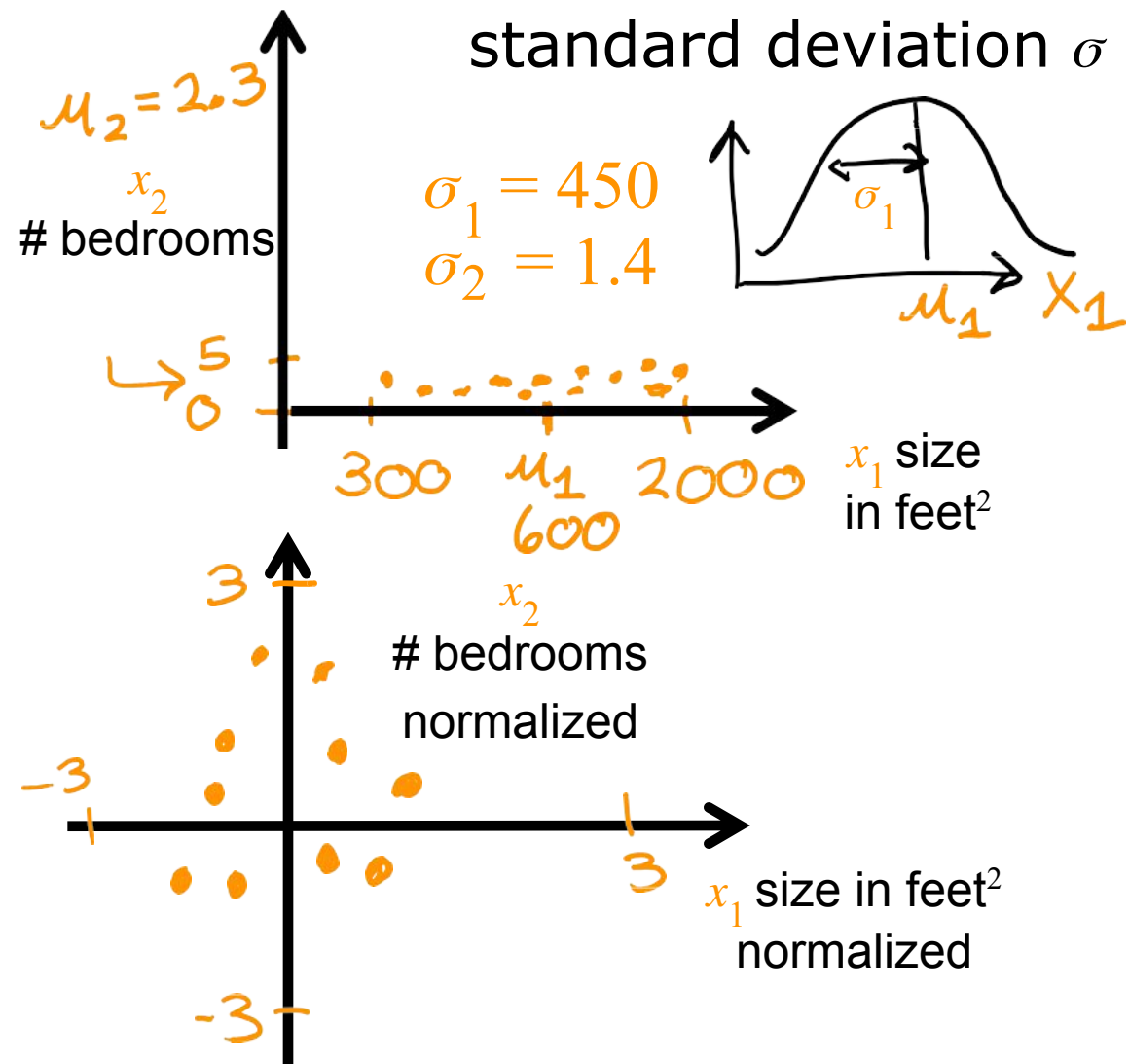
$$0 \leq x_2 \leq 5$$

$$x_2 = \frac{x_2 - \mu_2}{5 - 0}$$

max-min

$$-0.46 \leq x_2 \leq 0.54$$

# Z-score normalization



$$300 \leq x_1 \leq 2000$$

$$0 \leq x_2 \leq 5$$

$$x^1 = \frac{x_1 - \mu_1}{\sigma_1}$$

$$x^2 = \frac{x_2 - \mu_2}{\sigma_2}$$

$$-0.67 \leq x_1 \leq 3.1 \quad -1.6 \leq x_2 \leq 1.9$$

# Feature scaling

aim for about  $-1 \leq x_j \leq 1$  for each feature  $x_j$

$$-3 \leq x_j \leq 3$$

$$-0.3 \leq x_j \leq 0.3$$

} acceptable ranges

$$0 \leq x_1 \leq 3$$

$$-2 \leq x_2 \leq 0.5$$

$$-100 \leq x_3 \leq 100$$

$$-0.001 \leq x_4 \leq 0.001$$

$$98.6 \leq x_5 \leq 105$$

okay, no rescaling

okay, no rescaling

too large  $\rightarrow$  rescale

too small  $\rightarrow$  rescale

too large  $\rightarrow$  rescale



# Checking Gradient Descent for Convergence

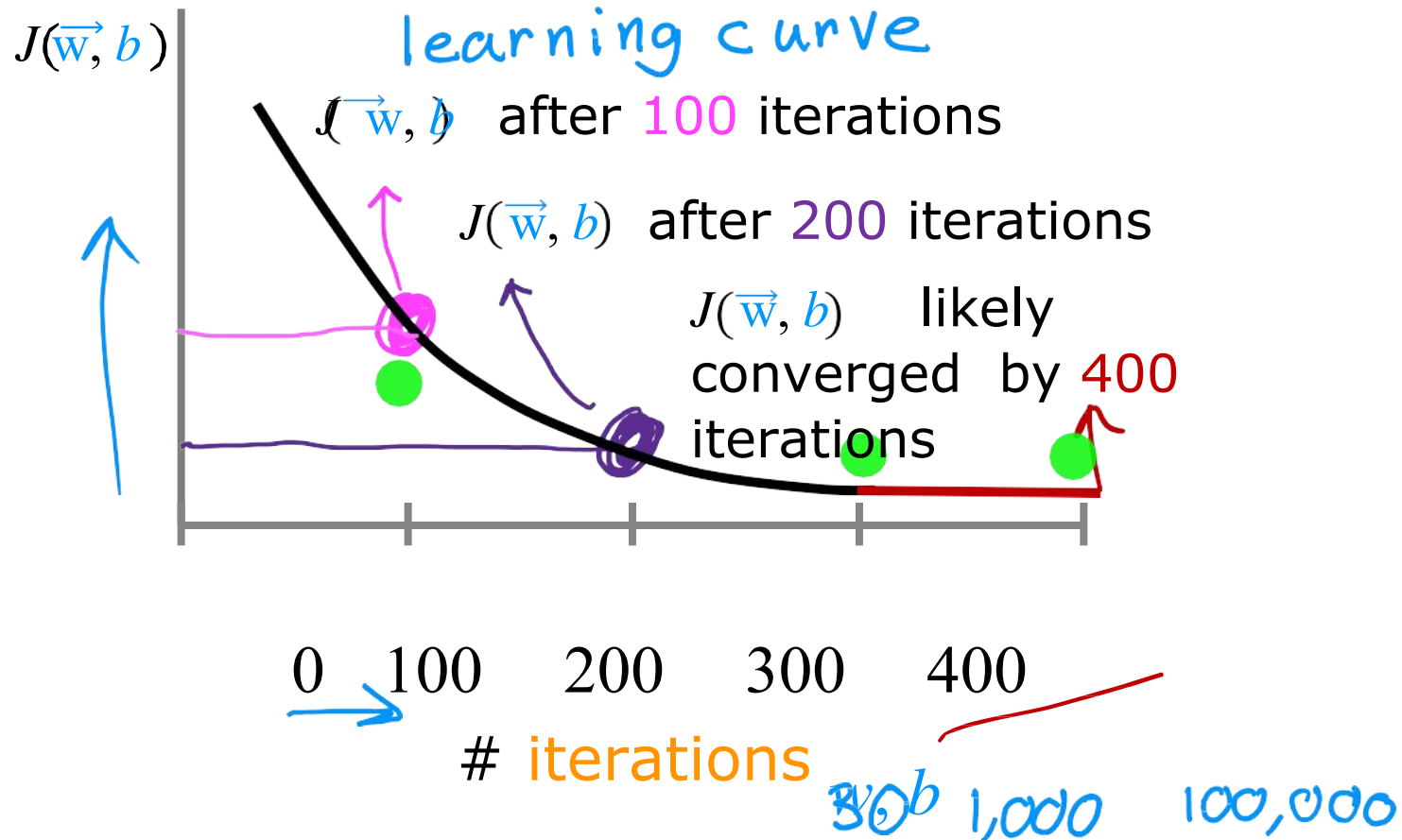
# Gradient descent

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

# Make sure gradient descent is working correctly

objective:  $\min_{\vec{w}, b} J(\vec{w}, b)$   $J(\vec{w}, b)$  should **decrease** after every iteration



# iterations needed varies

Automatic convergence test

Let  $\varepsilon$  "epsilon" be  $10^{-3}$ .

$0.001$

If  $J(\vec{w}, b)$  decreases by  $\leq \varepsilon$  in one iteration, declare **convergence**.

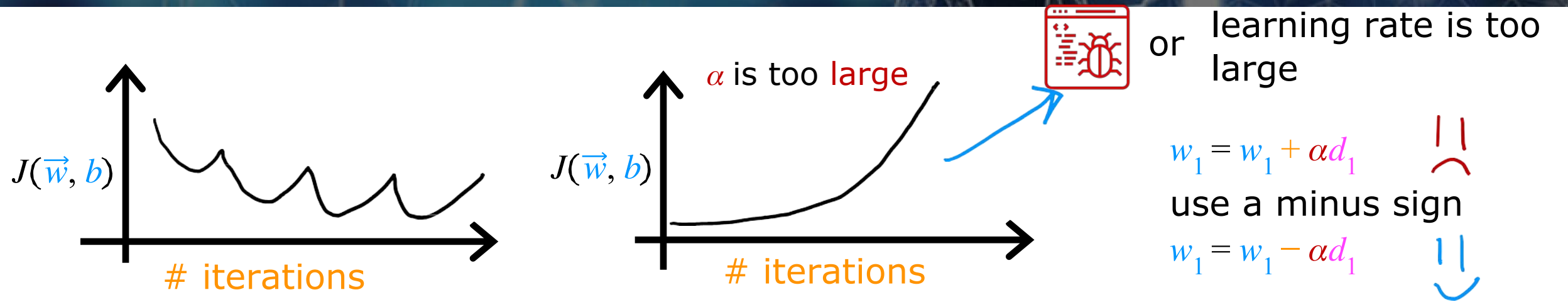
(found parameters  $\vec{w}, b$

to get close to global minimum)

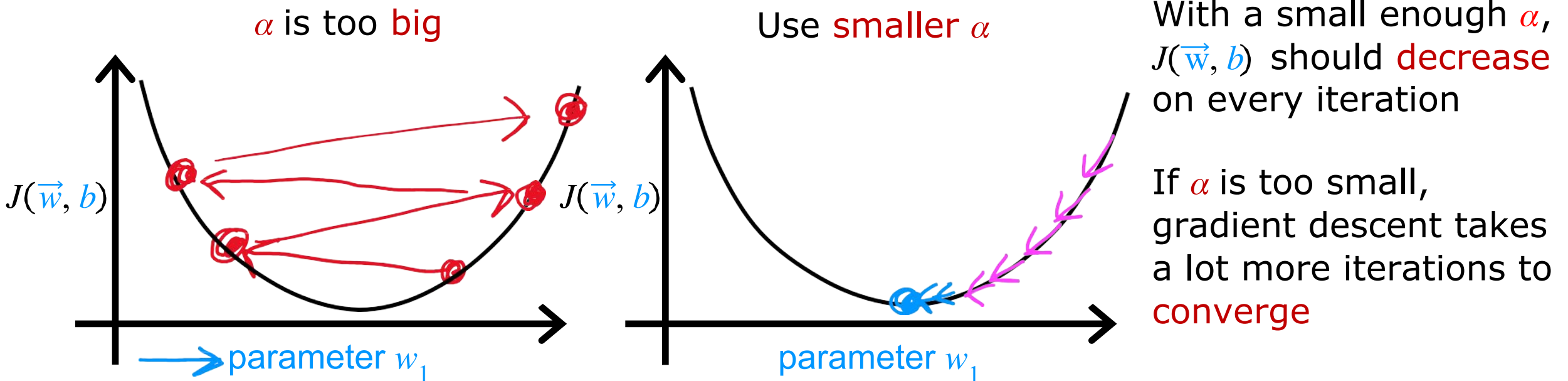


# Choosing the Learning Rate

# Identify problem with gradient descent

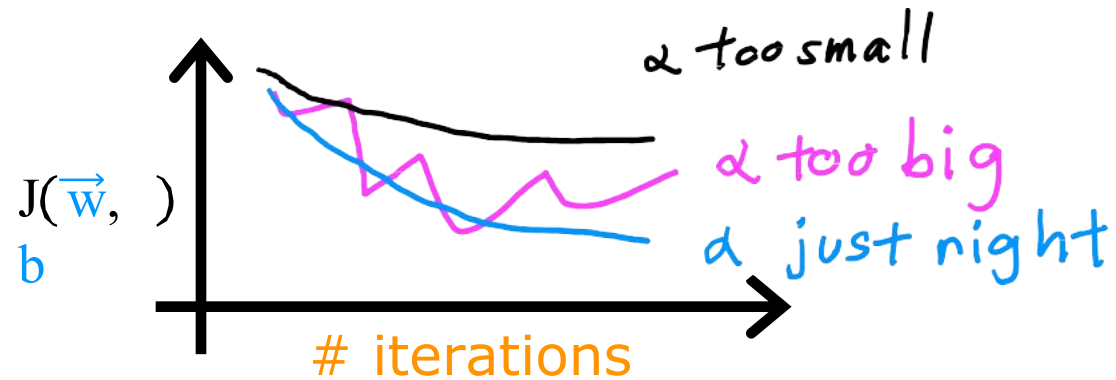
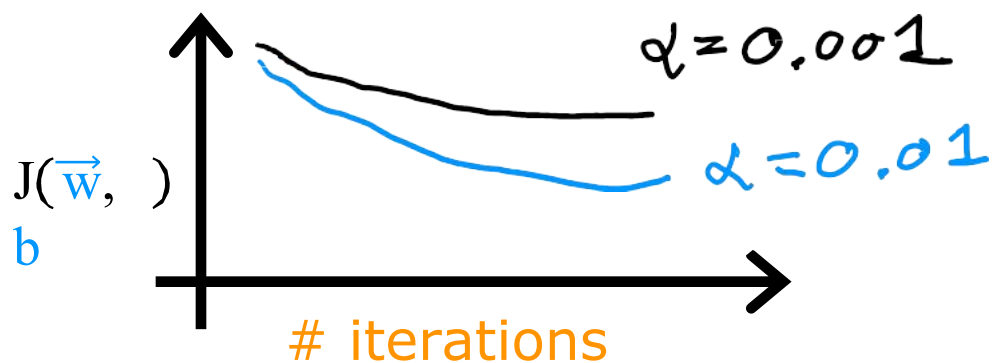


Adjust learning rate



Values of  $\alpha$  to try:

... 0.001 0.003 0.01 0.03 0.1 0.3 1  
     $\nearrow$        $\nearrow$        $\nearrow$        $\nearrow$        $\nearrow$        $\nearrow$   
     $3\times$        $\approx 3\times$        $3\times$        $\approx 3\times$        $3\times$        $\approx 3\times$





# Feature Engineering

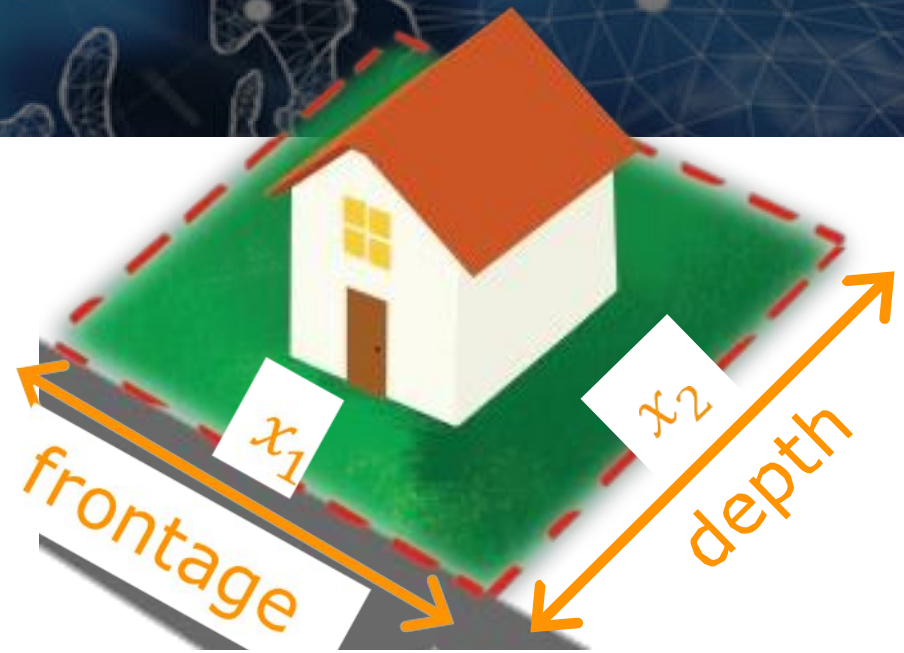
# Feature engineering

$$f_{w,b}(\vec{X}) = \underbrace{w_1 x_1}_{\text{frontage}} + \underbrace{w_2 x_2}_{\text{depth}} + b$$

$$\text{area} = \text{frontage} \times \text{depth}$$

$$x_{\text{new feature}} = x_1 x_2$$

$$f_{w,b}(\vec{X}) = \underbrace{w_1 x_1}_{\text{frontage}} + \underbrace{w_2 x_2}_{\text{depth}} + \underbrace{w_3 x_3}_{\text{area}} + b$$

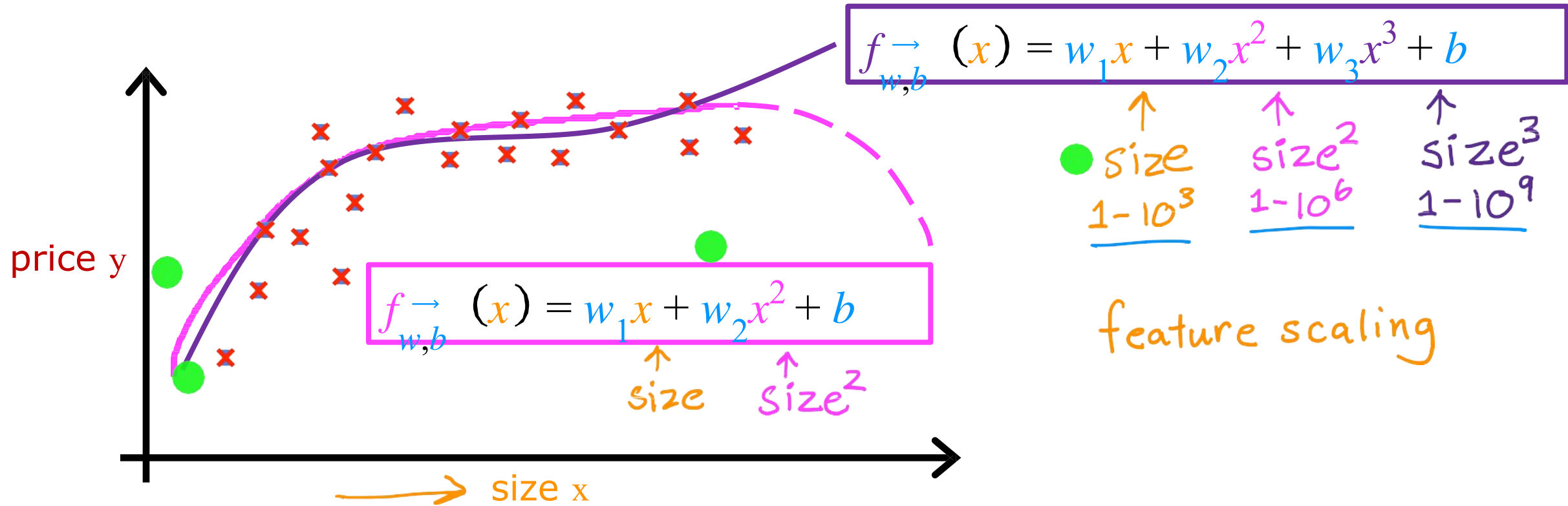


Feature engineering:  
Using **intuition** to design **new features**, by transforming or combining original features.

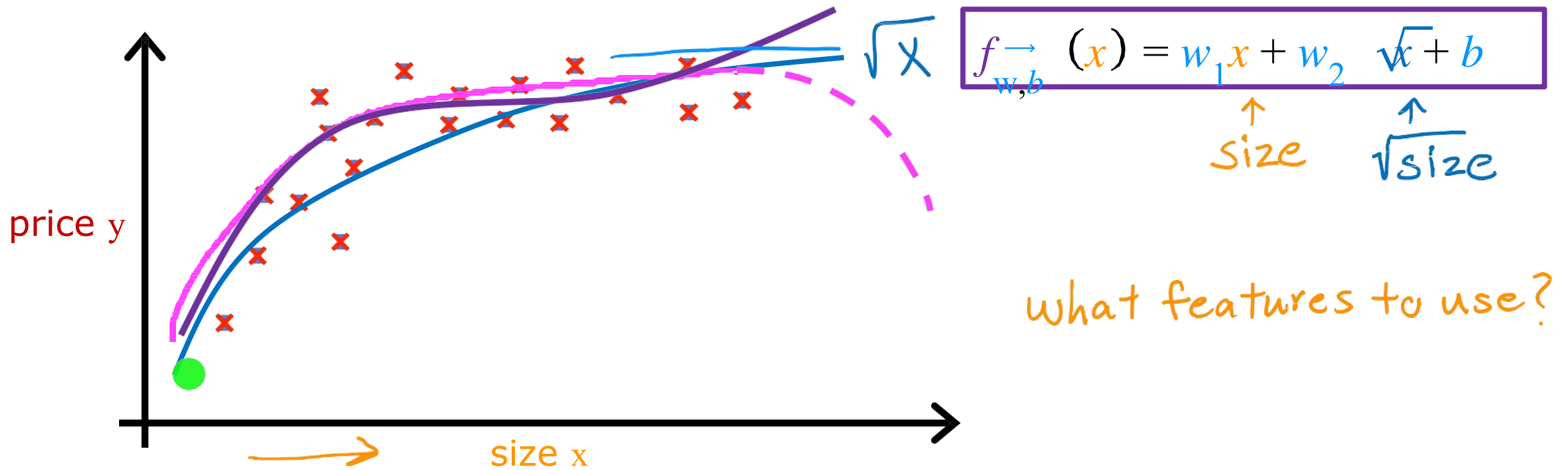


# Polynomial Regression

# Polynomial regression



# Choice of features





Q & A